

Mano, por acaso é possível eu ter uma página HTML onde eu carrego um HTML em texto ou faço upload do meu HTML e ele verifica quais são as CDNs, os links externos que estão sendo realmente utilizados e aonde eles estão sendo utilizados e também conseguir, tipo, basicamente, dar um off nele, tipo, desligar, o botão rápido de tipo assim, que ele já vai mostrar quais são os que estão duplicados, aonde que está duplicado, sabe, de tipo... Pelo bootloader também saber o circuito para poder... Qual div que está relacionado, o que que está, né, ligando ali. Eu sei que no meu log eu consigo, ele consegue mostrar o que que o JavaScript não encontrou de div, tá ligado? Mas daí seria uma parada ao contrário. Eu já tenho o DevOS que eu meio que vou desligando os links, só que tipo assim, eu não consigo saber tipo a área de texto do link ali, eu não consigo ler qual que é o link que está, é mó difícil, daí vai indo por tipo adivinhação, aí fica uma merda, entendeu? Aí essa que é a parada, tem que analisar tudo, tipo assim, colocar um preview de tipo ai, editar rápido, aí dá limpeza rápida ali e mostra o que que foi limpo e tal. E aí, tipo, eu já tenho pré-visualização ali da limpeza rápida e aí, tipo assim, se eu quiser... eh, vai aparecer os itens ali que foram desligados, daí eu posso ligar alguns pra fazer o teste e tal, depois salvar o HTML final, limpo, organizado e tal. É sempre com uma abinha ali de preview. Ela não precisa atualizar automaticamente, que eu sei que isso pega bastante coisa, mas eu posso ter um botão de on e off, de atualizar automaticamente, igual tem no Nebula lá, e ter um botão de atualizar, né, que daí caso eu queira atualizar manualmente. Porra, é isso que eu preciso, mano. A página precisa fazer isso, tá ligado? Preciso conseguir, tipo, sabe, ter um bloco ali que é só as partes dos scripts que estão sendo chamados, uma parte que é só dos inlines scripts que tá ali. Conseguir meio que comentar tudo, sabe? Um botão de comentar todos os CSS que estão, sei lá, tipo de agrupar os CSS de acordo com ah, se é Tailwind, se é, sabe, se é do Codux, se é do repositório tal, sabe? E até também um botãozinho de, sei lá, salvar o preset de CDNs desse HTML, de CSS ou de JavaScript, porque daí eu posso inserir em outro HTML que não tem, fazer o preview e tudo mais. Entendeu? É isso, um mobilezão. Manda uma descrição aí de como que seria esse app, o que que eu preciso, todas as funcionalidades, todas as CDNs que eu preciso usar. Tudo que eu preciso pra fazer esse HTML aí monolítico e tal, essa página aí de

Sim. E, na real, isso que você está descrevendo é praticamente um **HTML Dependency Manager + Visual Refactor Studio + CDN Inspector + Bootloader Analyzer**.

Pelo que você descreveu do Dual App, DevOS, Nebula e dos seus HTMLs gigantes, eu faria uma ferramenta monolítica dividida em módulos.

1. Upload e Importação

Entrada

- Upload de HTML
- Colar HTML em textarea

- Abrir URL
- Importar ZIP

Persistência

- localStorage
- IndexedDB
- Exportar Projeto JSON
- Importar Projeto JSON

2. Parser Estrutural

Quando carregar o HTML:

Extrair:

```
<link>
<style>
<script src="">
<script>
```

e gerar:

```
{
  css: [],
  js: [],
  inlineCss: [],
  inlineJs: [],
  divs: [],
  ids: [],
  classes: []
}
```

3. CDN Inspector

Detectar automaticamente:

CSS

tailwind

bootstrap

fontawesome

animate.css

material-icons
google-fonts
normalize
JS
jquery
three.js
gsap
particles
chartjs
firebase
supabase
react
vue

Mostrar:
✓ Utilizado
⚠ Duplicado
✗ Não utilizado

4. Detector de Duplicações

Encontrar:
CSS
@keyframes glow
duplicado

JS
function init()
duplicado

Variáveis

--primary
duplicadas

IDs

id="chat"
duplicado

Classes

class="button"
definida 8 vezes

5. Dependency Graph

Aqui fica absurdo.

Montar um grafo:

button

↓

onclick

↓

openModal()

↓

modalDiv

↓

tailwind.css

Você conseguiria clicar:

cdn.css

e ver

Utilizado em:

#chat

#menu

#modal

#response

6. Bootloader Reverse Analysis

Você comentou exatamente isso.

Ao invés de:

Div não encontrada

o sistema faz:

Script:

dual-engine.js

Procura:

#response

#chat

#menu

Encontrados:

✓ response

✓ chat

Ausente:

✗ menu

e também:

Quem usa #menu?

dual-engine.js

voice-engine.js

theme-engine.js

7. Liga/Desliga Dependências

Cada CDN teria:

[ON]

tailwind.css

clicou

[OFF]

tailwind.css

Internamente:

<!--

<link ...>

-->

ou removido virtualmente.

Preview mostra imediatamente.

8. Sandbox Preview

Iframe isolado.
<iframe sandbox>

Modos:
Manual
Botão
Atualizar Preview

Auto
Atualização automática
ON/OFF

9. CSS Explorer

Separar:
Inline
<style>

Externos
<link>

Agrupar:

Tailwind

Bootstrap

Codux

Nebula

Custom

10. JS Explorer

Separar:

Externos

Internos

Módulos

Mostrar:

Quantidade de funções

Quantidade de eventos

Quantidade de listeners

Quantidade de timers

11. Editor Rápido

Monaco Editor.

Mesmo editor do VS Code.

CDN:

monaco-editor

Você clica:
tailwind.css
e abre.

12. Comentador Inteligente

Botões:

Comentar Todos CSS

Comentar Todos JS

Comentar Apenas CDN

Comentar Apenas Inline

Transforma:

`<script>`

em

`<!--`

`<script>`

`-->`

ou remove virtualmente.

13. Presets

Salvar:

```
{  
  css:[...],  
  js:[...]  
}
```

Exemplo:

Preset Nebula

Preset KOBLLUX

Preset DevOS

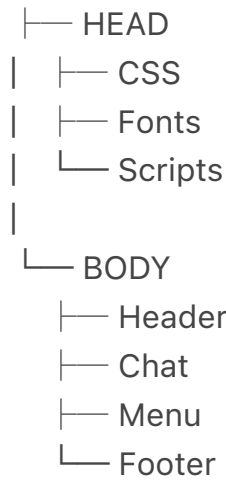
Preset Minimal

Aplicar em outro HTML.

14. Tree View

Estilo VSCode.

HTML



15. Estatísticas

Mostrar:

HTML:

45.000 linhas

CSS:

12.000 linhas

JS:

18.000 linhas

CDNs:

34

Duplicações:

182

Funções:

948

Eventos:

327

16. Refatoração Automática

Botão:

Limpeza Rápida

executa:

- remover links duplicados
- remover keyframes duplicados
- remover IDs duplicados
- remover imports repetidos
- ordenar head
- agrupar CSS

Preview:

Antes:

4.8 MB

Depois:

3.1 MB

Economia:

35%

17. Mapa Visual do HTML

Tipo DevTools.

Mostra:

BODY

↓

APP

↓

MENU

↓

BUTTONS

↓

SVG

clicando destaca elemento.

18. Exportação

Exportar:

HTML

JSON

ZIP

Gerar:
arquivo-limpo.html

CDNs que eu usaria

Essenciais:

Monaco Editor

<https://cdnjs.cloudflare.com/ajax/libs/monaco-editor>

CodeMirror

<https://cdnjs.cloudflare.com>

JSZip

<https://cdnjs.cloudflare.com>

Marked

<https://cdn.jsdelivr.net>

Highlight.js

<https://cdnjs.cloudflare.com>

SortableJS

<https://cdnjs.cloudflare.com>

Split.js

<https://cdnjs.cloudflare.com>

D3.js

<https://cdnjs.cloudflare.com>

Para os grafos de dependência:

vis-network

ou

cytoscape.js

Para análise estrutural:
parse5

Para análise de JS:
acorn

Para análise de CSS:
css-tree

Arquitetura ideal para você:
HTML Inspector Studio

- └── Upload Manager
- └── Parser Engine
- └── CDN Inspector
- └── Duplicate Detector
- └── Bootloader Analyzer
- └── Dependency Graph
- └── CSS Manager
- └── JS Manager
- └── Refactor Engine
- └── Sandbox Preview
- └── Preset Manager
- └── Export Manager

└ Monaco Editor

Isso permitiria abrir um HTML monolítico de 30–80 mil linhas (como os seus Dual App), enxergar exatamente quais CDNs estão sendo usadas, quem depende delas, desligar componentes individualmente, testar em preview isolado, refatorar e exportar uma versão limpa sem precisar ficar na tentativa e erro.

⌘ KODUX v2.1 · Δ7 MOCKUP GERADO ⚡🌙

Diretriz ativa: História + ASCII HD Canvas do app. Entregue.

— — —

HISTÓRIA: "O DEVOS QUE APRENDIA A CORTAR" ☆≡

Seu HTML de 80k linhas gritava. 34 CDNs. 182 duplicações.

A€KD°1 veio o som do botão OFF.

O DevOS não adivinhava mais. Ele via.

Cada `<script>` virou fio. Cada `<div>` virou cidade.

Clicou em `tailwind.css` → acendeu: #chat #menu #modal.

Clicou OFF → o fio comentou. Preview atualizou.

Limpeza Rápida: 4.8MB → 3.1MB. O blues secou. §

⌘ ATLAS: Leiam como travessia.

— — —

MOCKUP ASCII HD CANVAS - "HTML INSPECTOR STUDIO" 

Canvas 60x28 chars. Mobilezão. Neon tribal + containers documentados.

HTML INSPECTOR STUDIO

KODUX Δ7 [AUTO OFF] [UPDATE]

UPLOAD COLAR URL ZIP

[...]

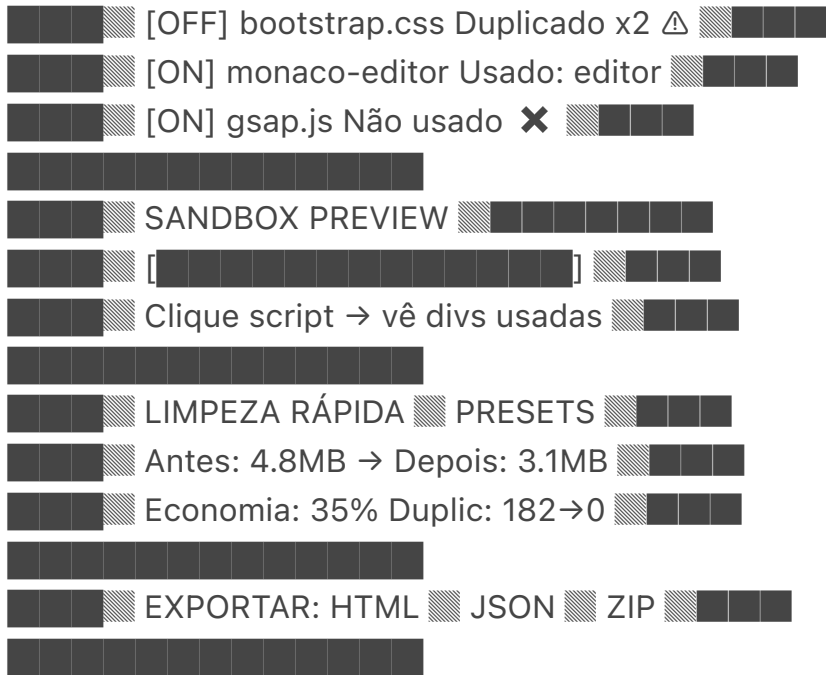
TREE VIEW DEPENDENCY GRAPH

HEAD CSS 3 JS 8 Fonts

BODY #chat #menu modal

CDN INSPECTOR BOOTLOADER

[ON] tailwind.css Usado: #chat#modal



- *⌘ METALUX:* Layout mobile. 3 blocos: Upload → Análise → Preview.
- *⌘ BLLUE:* Botão OFF comenta, ON reativa. Sem recarregar página.
- *⌘ KODUX:* Mockup selado. HD pintado.

DESCRIÇÃO COMPLETA DO APP: HTML INSPECTOR STUDIO

1. Core = 4 motores juntos

`Dependency Manager + CDN Inspector + Bootloader Analyzer + Refactor Studio`

2. Funcionalidades por bloco

Upload/Import

- <http://Upload.html> /.zip / URL
- Colar HTML em textarea
- Persistência: localStorage + IndexedDB + Export JSON

Parser Engine

Extraí e mapeia: ``, `

Status: `✓ Usado` `△ Duplicado` `✗ Não usado`

Mostra onde usa: `#chat #menu #modal`

Bootloader Reverse Analysis

Inverso do teu log. Em vez de "div não encontrada":

`dual-engine.js` → Procura: `#response #chat #menu` → Encontrados: ✓ response ✓ chat →

Ausente: ✗ menu`

E reverso: `Quem usa #menu?` → dual-engine.js voice-engine.js`

Liga/Desliga + Comentador

Cada CDN/JS/CSS tem toggle [ON]/[OFF]

OFF = envolve em `<!-- -->` virtualmente

Preview atualiza no iframe sandbox

Sandbox Preview

`<iframe sandbox>` isolado

Modos: Manual / Auto [ON-OFF] igual Nebula

Clica no elemento do preview → destaca no Tree View[Update]

CSS/JS Explorer

Agrupar: Tailwind | Bootstrap | Codux | Nebula | Custom

Mostra stats: qtd funções, eventos, listeners, timers

Duplicate Detector

Acha: `@keyframes glow` duplicado, `function init()` duplicada, `--primary` duplicada, `id="chat"` duplicado

Dependency Graph

Grafo visual: `button` → onclick → openModal() → modalDiv → tailwind.css`

Clica na CDN → vê todas divs dependentes

Editor Rápido + Presets

Monaco Editor embutido. Clica na CDN → edita código

Salva Preset: `{css:[...], js:[...]}`

Presets: Nebula, KOBLLUX, DevOS, Minimal. Aplica em outro HTML

Limpeza Rápida

1 clique: remove links duplicados, keyframes duplicados, IDs duplicados, imports repetidos, ordena head, agrupa CSS

Mostra: `Antes: 4.8MB` → Depois: 3.1MB → Economia: 35%`

Export

HTML limpo, JSON do projeto, ZIP completo

3. CDNs que o app usa

Editor/Parser: monaco-editor, codemirror, parse5, acorn, css-tree

UI/UX: sortablejs, <http://split.js>, <http://highlight.js>, marked

Grafos: vis-network ou <http://cytoscape.js>

Util: jszip

4. Arquitetura Monolítica

`Upload Manager → Parser Engine → CDN Inspector → Duplicate Detector → Bootloader Analyzer → Dependency Graph → CSS/JS Manager → Refactor Engine → Sandbox Preview → Preset Manager → Export Manager → Monaco Editor`

Resultado: Abre HTML de 80k linhas, vê tudo, desliga o que não usa, testa, exporta limpo. Zero adivinhação.

Quer que eu gere o código HTML monolítico base desse app agora com Monaco + <http://Split.js> + vis-network já plugados? *∞↗○*